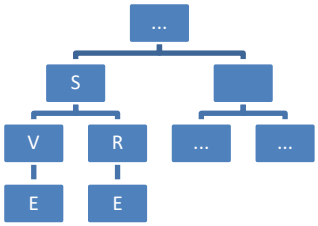
Obiective

- Testarea aplicaţiilor la diferite niveluri de testare, e.g., testare unitară, testare de integrare.
- Utilizarea tool-urilor folosite în procesul de testare (TestLink, Jenkins, JUnit, Maven, Git, Mockito).

Cerinţe

Să se realizeze următoarele task-uri:

Task, puncte	Descriere task
	Identificaţi la nivelul arhitecturii aplicației clase particulare, codificate aici prin E, R, V şi S care înde plinesc următoarele condiţii: *E reprezintă o entitate din domeniul problemei, V este un validator pentru elemente de tip E, R este un repository cu elemente de tip E, iar S permite gestionarea repository-ului, folosind validatorul.* Considerăm în aplicaţia dezvoltată existenţa următoarei diagrame de dependenţă între clasele E, V, R şi S. 
[Unit Testing. JUnit. Mockito] **2 puncte**	Realizaţi testarea în izolare (unit testing) pentru clasa S, utilizând obiecte **mock** pentru alte două clase dependente. Testarea în izolare va folosi framework-urile **JUnit** (vezi **Tutorial JUnit** în Lab02) şi **Mockito** (vezi **Tutorial Mockito**). Fiecare echipă va alege unul dintre scenariile de mai jos. (1) V <--- S ---> R și ulterior integrare top-down, breadth first (2) S ---> V ---> E și ulterior integrare top down, depth first (3) S ---> R ---> E și ulterior integrare top down, depth first Clasa de testare pentru S va avea **minim 2 teste**. Testele vor evidenția atât evaluarea cu `assert` cât și `verify` (vezi **Tutorial Mockito**). **Nu se cer următoarele:** • elaborarea de fişiere similare cu **Lab02_BBT_TCs_Form.xls** şi **Lab03_WBT_TCs_Form.xls**; • crearea *suplimentară* în TestLink a suite-lor de teste pentru testarea unitară a modulelor E, V, R şi S. **Observație:** De obicei, nu folosim obiecte mock pentru clase din pachetele domain/model, i.e., clasele de tip E. În mod particular pentru Lab04, pentru a evidenția integrarea top down depth first, vom folosi obiecte mock pentru clasele de tip E.
[Integration Testing. Mockito] **2 puncte**	Realizaţi testarea de integrare (integration testing) utilizând strategia de **integrare incrementală top-down** pentru scenariul ales. Se vor evidenţia următoarele clase cu teste: Exemplu pentru (1): ○ **Step 1.** testare unitară S. ***[task-ul anterior]*** ○ **Step 2.** integrare V (se testează S cu V; pentru R se folosesc obiecte *mock*; opțional, pentru E se pot folosi mock-uri). ○ **Step 3.** integrare R (se testează S + V cu R; opțional, pentru E se pot folosi mock-uri). ○ **Step 4.** integrare E (se testează S +V + R cu E), dacă la pașii anteriori s-au folosit mock-uri pentru E. Exemplu pentru (2): ○ **Step 1.** testare unitară S. ***[task-ul anterior]*** ○ **Step 2.** integrare V (se testează S cu V; pentru E se folosesc obiecte *mock*; opțional, pentru R se pot folosi mock-uri). ○ **Step 3.** integrare E (se testează S + V cu E; opțional, pentru R se pot folosi mocl-uri). ○ **Step 4.** integrare R (se testează S +V + E cu R), dacă la pașii anteriori s-au folosit mock-uri pentru R. Fiecare clasă de test de la **Step 2, Step 3** şi **Step 4** va avea **minim 2 teste**. **Step 4** nu este obligatoriu.
[TestLink] **2 puncte**	În cadrul proiectului **ProiectAAA,** corespunzător userului **xyir1234** utilizat anterior pentru Lab03, se vor realiza următoarele task-uri: 1.1. definiţi planul de testare **xyir1234_IntT_TP** în cadrul proiectului **ProiectAAA** (secţiunea *Test Plan*); 1.2. creaţi suita de teste **xyir1234_IntT** care va conţine **3-4 cazuri de testare**, câte un caz de testare pentru fiecare pas de aplicare a strategiei de integrare **Top down** (secţiunea *Test Specification*);

	1.3. asociaţi cazurile de testare create la planul **xyir1234_IntT_TP**; 1.4. asociaţi cazurile de testare create la cerinţele create anterior, după caz, la **xyir1234_F01** sau **xyir1234_F02**; 1.5. generaţi documentaţia aferentă din (secţiunea *Test Specification*, opţiunea *Generate Test Specification Document*) în format .docx.
[Jenkins] **2 puncte**	Se va crea, configura şi executa **câte un job** (vezi **Tutorial Jenkins**) pentru fiecare plan de testare creat anterior, i.e., **xyir1234_BBT_TP**, **xyir1234_WBT_TP**, **xyir1234_IntT_TP.** Job-urile create vor fi denumite **xyir1234Job_BBT**, **xyir1234Job_WBT** şi **xyir1234Job_IntT**. **Observaţie:** •pentru fiecare test creat în TestLink se va verifica (1) dacă este un test executat *automat* şi (2) dacă are setat corect *numele clasei de test* şi *numele metodei de test* (vezi **Tutorial TestLink**, **pagina 14**).
[TestLink+ Jenkins] **1 punct**	Vizualizarea rezultatului execuţiei fiecărui job Jenkins în TestLink (vezi **Tutorial Jenkins**). Pentru fiecare test case executat este necesar să apară statusul **passed**.
[Git] **1 punct**	Se va actualiza conţinutul repository-ului **Git** cu documentele elaborate în cadrul acestei teme: • în folderul **Docs/Lab04** fişierul cu documentaţia generată în TestLink; • pachetul/ele cu teste implementate în Java pentru Lab04; • dacă este cazul, codul sursă modificat în urma depanării.
[Chestionar] **2 puncte**	Înainte de predarea temei de laborator, studenții sunt invitați sa completeze un chestionar cu întrebări referitoare la instrumentele AI utilizate pentru rezolvarea task-urilor incluse în tema de laborator**.** *Link-ul la chestionar este disponibil în postarea* **Link-uri importante***, în Teams.* **Fiecare student din echipă care completează chestionarul și predă tema de laborator la timp primește nota 12 pentru Lab04. Tema de laborator predată cu întârziere este depunctată cu minim 2 puncte, dar se acordă cele 2 puncte aferente chestionarului, dacă studentul l-a completat anterior predării temei de lab.**

Timp de lucru recomandat pentru rezolvarea temei de laborator ~ 4 ore/echipă.

Predarea temei de laborator

[Unit Testing. JUnit. Mockito]
- implementarea claselor cu teste pentru clasa S.

[Integration Testing. Mockito]
- implementarea claselor cu teste folosind strategia de integrare **Top Down (depth first** sau **breadth first)**.

[TestLink+Jenkins]
- Câte un job pentru fiecare dintre cele trei planuri de testare create în TestLink: **xyir1234Job_BBT**, **xyir1234Job_WBT** şi **xyir1234Job_IntT.**
- Fiecare test din TestLink are statusul **passed**.

Termene de predare

Săptămâna	**Tema de laborator**			**Primul termen** de predare	**Ultimul termen** de predare
S07	**L04.**	Niveluri de testare	JUnit, TestLink, Jenkins, Git, Mockito	S09	S11*°
S08				S10	S12*°

***) Temele restante se vor putea preda în limita timpului disponibil.**
°) Se pot preda cel mult două teme de laborator.